

Data Project — Zynq DDR Data Logger

FPGA Zybo Z7-20

HDL Verilog

Tool Vivado

SoC Zynq-7000

Status In Development

Data Project הפרויקט מדגים **Zybo Z7-20** המפותח בצורה הדרגתית על כרטיס FPGA / SoC הוא פרויקט העברה, העברה, FIFO-אחסון ב, Packet דרך יצירת, Verilog-מותאמים אישית ב RTL מסלול נתונים מלא, החל ממודולי Vitis C. ועד אימות הנתונים באמצעות, DDR וזיכרון AXI DMA אינטגרציה עם, AXI-Stream באמצעות

תוכן עניינים

- סקירת הפרויקט
- כלים ופלטפורמה
- תרשים בלוקים של המערכת
- ארכיטקטורת המערכת
- מאפייני התכנון המרכזיים
- Data Packet-מבנה ה
- שלבי הפיתוח
- מבנה הפרויקט
- תיעוד
- תוכנית האימות
- המטרה-הסופית
- יוצר-הפרויקט

סקירת הפרויקט

(**Data Logger**) מלאה לרישום והעברת נתונים FPGA / SoC מטרת הפרויקט היא לבנות מערכת

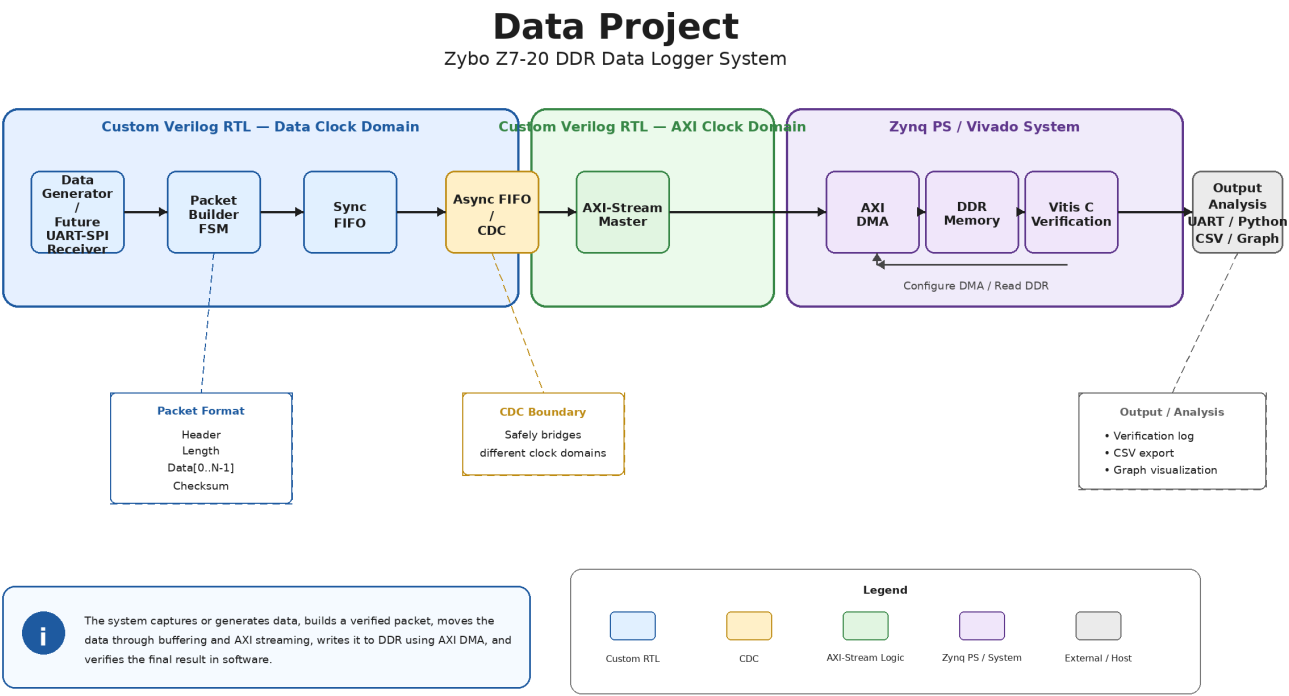
-בעל מבנה מוגדר, נשמרים ב Packet הנתונים נארזים בתוך. FPGA-המערכת מתחילה ממקור נתונים מבוקר בתוך ה לבסוף, הנתונים יאומתו. AXI DMA באמצעות DDR ובהמשך נכתבים לזיכרון AXI-Stream מועברים באמצעות, FIFO Zynq Processing System-בתוך ה ARM-באמצעות תוכנה הרצה על מעבד ה

הפרויקט נבנה בצורה הדרגתית ומודולרית, כאשר כל בלוק נבנה ונבדק בנפרד לפני חיבורו למערכת המלאה.

כלים ופלטפורמה

- Target Board:** Zybo Z7-20
- FPGA / SoC:** Xilinx Zynq-7000
- Hardware Design Tool:** Vivado
- Hardware Description Language:** Verilog HDL
- Software Tool:** Vitis C
- Version Control:** Git / GitHub
- Future Data Analysis:** Python / CSV / Graph

תרשים בלוקים של המערכת

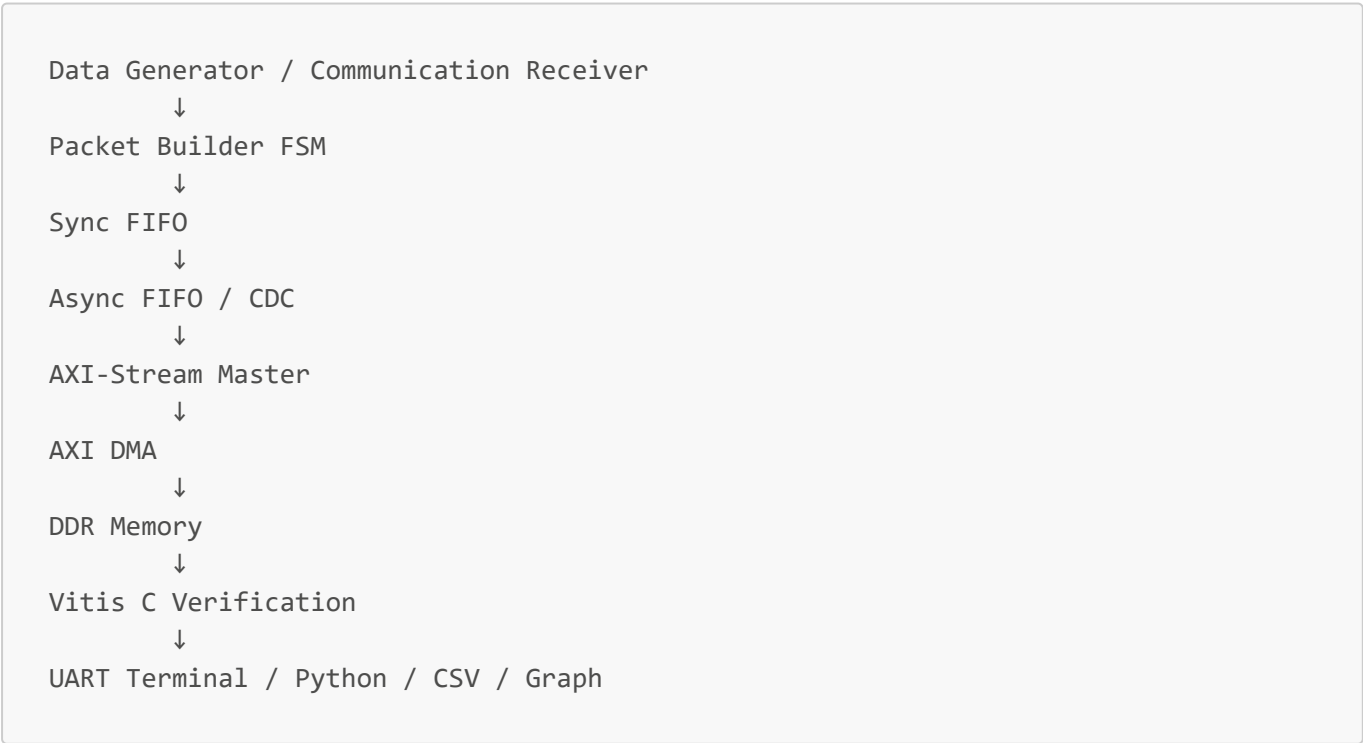


High-level architecture of the planned Data Project system.

המימוש מתבצע בהדרגה בהתאם. Data Project-התרשים מציג את הארכיטקטורה הסופית המתוכננת של ה לשלבי הפיתוח של הפרויקט.

ארכיטקטורת המערכת

ארכיטקטורת המערכת הסופית המתוכננת היא:



פנימי **Data Generator**-במימוש הראשוני נעשה שימוש ב

SPI או **UART** בשלב מתקדם יותר ניתן יהיה להחליף את מקור הנתונים הפנימי בממשק תקשורת אמיתי, לדוגמה

מאפייני התכנון המרכזיים

1. Modular RTL Design

נפרדים RTL המערכת בנויה ממודולי

עצמאי Verification משלו ותהליך Testbench, לכל בלוק תפקיד מוגדר

המרכזיים כוללים RTL-מודולי ה

- Data Generator
- Packet Builder FSM
- Sync FIFO
- AXI-Stream Master
- Async FIFO / CDC
- Full RTL Integration Testbench
- Custom IP blocks נפרדים עבור Vivado Block Design

2. Structured Packet Format

הפרויקט אינו מעביר ערכים אקראיים ללא מבנה

הנתונים נארזים במבנה מוגדר

```
Header → Length → Data Words → Checksum
```

ובתוכנה DDR מבנה זה מאפשר לעקוב אחר הנתונים ולאמת אותם בצורה ברורה בסימולציה, ובהמשך בזיכרון

3. Testbench-Based Verification

ייעודי לפני חיבורו לבלוק הבא Testbench נבדק בנפרד באמצעות RTL כל מודול

Full RTL Integration Testbench-לאחר השלמת הסימולציות של המודולים הבודדים, המודולים שנבדקו חוברו ב

בבדיקה זו אומת מסלול הנתונים המלא

```
Data Generator
  ↓
Packet Builder FSM
  ↓
Sync FIFO
```

↓
AXI-Stream Master

ה-Vivado Block Design ול-IP Packaging עבר בהצלחה לפני המעבר ל-Full RTL Integration.

בצורה הדרגתית Integration גישה זו מאפשרת לבודד תקלות בשלבים מוקדמים ולבצע

4. Custom IP Packaging and Block Design

נפרדים Vivado Custom IP blocks נארזו כ-RTL מודולי ה-RTL Verification, לאחר השלמת ה

ה-Vivado Block Design וחוברו ב-IP Repository נוספו ל-Custom IPs ה-

ה-Synthesis והמערכת עברה HDL Wrapper נוצר, Validation עבר Block Design ה-

גישה זו מאפשרת להציג את ארכיטקטורת המערכת בצורה ברורה ולחבר את הלוגיקה המותאמת אישית למערכת ה-Zynq.

5. AXI-Stream and AXI DMA Integration

ה-AXI DMA לכיוון ה-FIFO שפותח בפרויקט אחראי להעברת המידע מה-AXI-Stream Master ה-

ה-DDR וזיכרון ה-Zynq Processing System ה-AXI DMA, ה-Custom RTL בשלב הנוכחי מתבצעת אינטגרציה בין ה

ה-Bצורה יעילה, ללא צורך בכך שה DDR לזיכרון Programmable Logic מאפשר להעביר את הנתונים מה AXI DMA יעתיק כל מילה בנפרד CPU.

6. DDR and Software Verification

ה-DDR יישמר בזיכרון ה-Packet ה-DDR, וה-AXI DMA לאחר השלמת אינטגרציית ה

ותאמת DDR buffer תקרא את ה Vitis C תוכנית:

- Header
- Length
- Data order
- Checksum
- PASS / FAIL result

7. Clock Domain Crossing

ה-Clock Domain Crossing לצורך Async FIFO התכנון יורחב באמצעות, ה-DDR וה-DMA לאחר השלמת מסלול ה

שונים Clock Domains שלב זה יאפשר עבודה נכונה בין חלקים במערכת הפועלים עם.

Data Packet-מבנה ה

פשוט Packet המערכת משתמשת במבנה:

```
Word 0: Header
Word 1: Length
Word 2: Data[0]
Word 3: Data[1]
...
Word N: Data[N-1]
Word N+1: Checksum
```

דוגמת הבדיקה הראשונית:

```
Header    = 0xA5A50001
Length    = 6
Data      = 1, 7, 8, 3, 6, 3
Checksum  = 28 = 0x0000001C
```

הצפוי Packet-ה:

```
Word 0: 0xA5A50001
Word 1: 0x00000006
Word 2: 0x00000001
Word 3: 0x00000007
Word 4: 0x00000008
Word 5: 0x00000003
Word 6: 0x00000006
Word 7: 0x00000003
Word 8: 0x0000001C
```

המלא מתועד בקובץ Protocol-ה:

```
docs/data_protocol.md
```

שלבי הפיתוח

הפרויקט מפותח בהתאם לשלבים הבאים:

Stage	Name	Status
0	GitHub + Project Structure	✓ Done
1	Data Protocol	✓ Done
2	Data Generator + Testbench	✓ Done
3	Packet Builder FSM + Testbench	✓ Done

Stage	Name	Status
4	Sync FIFO + Testbench	✓ Done
5	AXI-Stream Master + Testbench	✓ Done
6	Full RTL Integration Simulation	✓ Done
7	Custom IP Packaging + Vivado Block Design	✓ Done
8	AXI DMA + Zynq PS + DDR Hardware Integration	🔧 In Progress
9	Vitis C Verification	🕒 Planned
10	Async FIFO + CDC + Testbench	🕒 Planned
11	Full Integration with CDC + DMA + DDR	🕒 Planned
12	ILA + Vivado Reports	🕒 Planned
13	Python CSV / Graph	🕒 Planned
14	UART or SPI Extension	🕒 Planned
15	Portfolio Polish	🕒 Planned

שבתיקיית התיעוד Project Checklist-למעקב מפורט אחר ההתקדמות ניתן לעיין ב

Current Progress: שלבים 0-7 הושלמו במלואם. הפרויקט נמצא כעת ב **Stage 8 — AXI DMA + Zynq PS + DDR Hardware Integration**.

מבנה הפרויקט

```
Data_Project/
├── docs/           Project documentation, work plan, checklist and protocol
├── images/        Block diagrams, waveforms and screenshots
├── ip_repo/       Vivado custom IP repository
├── python/        Python scripts for CSV and graphs
├── reports/       Timing, utilization and simulation reports
├── rtl/           Verilog RTL source files
├── tb/           Testbench files
├── vitis/         Vitis C software
├── vivado/        Vivado project files and block design notes
└── README.md
```

תיעוד

קבצי התיעוד המרכזיים:

- **Work Plan:** docs/work_plan.md
- **Data Protocol:** docs/data_protocol.md

- **Project Checklist:** docs/project_checklist.md
- **Hebrew Checklist PDF:** docs/data_project_checklist_hebrew.pdf
- **English Checklist PDF:** docs/data_project_checklist_english.pdf
- **Block Diagram:** images/data_project_block_diagram.png
- **Full Project Plan:** docs/work_plan_he.pdf

לצורך תיעוד Repository-נשמרים כחלק מה Vivado ודוחות Waveforms, screenshots, בנוסף, תוצאות סימולציה תהליך הפיתוח והבדיקות.

תוכנית האימות

הפרויקט נבדק במספר רמות:

Behavioral Simulation

ייעודי Testbench נבדק בנפרד באמצעות RTL כל מודול

השלב הושלם עבור:

- Data Generator
- Packet Builder FSM
- Sync FIFO
- AXI-Stream Master

Full RTL Simulation

Vivado Block Design, AXI DMA I-DDR, לפני המעבר ל Full RTL Integration Simulation בוצעה

בבדיקה חוברה השרשרת:

```
Data Generator
  ↓
Packet Builder FSM
  ↓
Sync FIFO
  ↓
AXI-Stream Master
```

Header, Length, שישה Data Words ו-Checksum מלא בן 9 חילים, כולל Packet הסימולציה אימתה.

Packet-מופעל על המילה האחרונה של ה `axis_tlast` בנוסף אומת כי

הסתיים בהצלחה Full RTL Integration-ה.

Vivado Custom IP and Block Design

נפרד Vivado Custom IP-שנבדק נארז כ RTL כל מודול

Synthesis והמערכת עברה HDL Wrapper נוצר, Validate Design בוצע, Block Design-חוברו ב Custom IP blocks-ה

Vivado Hardware Integration

שלב זה נמצא כעת בתהליך.

Hardware Integration לצורך Vivado ב-DDR וזיכרון AXI DMA Zynq PS משולב עם Custom AXI-Stream output ה-I-Bring-Up על כרטיס ה-Zybo Z7-20.

Vitis C Verification

Packet-ותאמת את ה-DDR buffer תקרא את ה-DMA-תשלוט ב C לאחר השלמת אינטגרציית החומרה, תוכנית שהתקבל.

Hardware Debug

בזמן ריצה על חומרה אמיתית FPGA-לצורך בדיקת אותות פנימיים של ה ILA בהמשך ישולב.

Python Visualization

Graphs. והפקת CSV ישמש בהמשך לייצוא נתונים, יצירת Python.

המטרה הסופית

ואימות DDR ועד לזיכרון RTL החל מתכנון, FPGA / SoC, המטרה הסופית היא להדגים מסלול נתונים מלא במערכת באמצעות תוכנה.

הפרויקט מיועד להדגים יכולת ב:

- Custom Verilog RTL תכנון
- Testbenches כתיבת
- FSM-based control logic בניית
- FIFO buffering שימוש ב
- AXI-Stream עבודה עם
- Vivado Custom IP כ-RTL של Packaging
- Vivado Block Design בנייה ואימות של
- AXI DMA ו-DDR אינטגרציה עם
- Vitis C verification software כתיבת
- Clock Domain Crossing עבודה עם
- Hardware Debug לצורך ILA-שימוש ב
- Git / GitHub תיעוד וניהול פרויקט בצורה מסודרת באמצעות

סיכום

מלאה Data Logger בסיסיים למערכת RTL מובנה המתפתח בהדרגה ממודולי FPGA / SoC הוא פרויקט Data Project על Zynq המבוססת על.

ולתעד בצורה ברורה כל Integration הדגש בפרויקט הוא לא רק לגרום למערכת לעבוד, אלא גם להבין, לבדוק, לבצע שלב בתהליך הפיתוח.

יוצר הפרויקט

Created by **Dvir Gedanken** as an FPGA / SoC portfolio project.